컴파일러 스터디 3

2026 김은빈

Back-patching

목표 결과

문 앞에 섰다.
문을 열려면 45쪽으로 가시오.
돌아가려면 30쪽으로 가시오.

Back-patching

중간 단계

문 앞에 섰다.
문을 열려면 9999쪽으로 가시오.
돌아가려면 9999쪽으로 가시오.

Back-patching

중간 단계

문 앞에 섰다.
문을 열려면 45쪽으로 가시오.
돌아가려면 9999쪽으로 가시오.

Back-patching

중간 단계

문 앞에 섰다.
문을 열려면 45쪽으로 가시오.
돌아가려면 30쪽으로 가시오.

RustPython 조건문 구현

책에서 OpJumpNotTruthy: True가 아니면(False면) 특정 주소로 점프

CPython, RustPython에선
“POP_JUMP_IF_FALSE 3” 처럼 함 (숫자 칸은1byte)

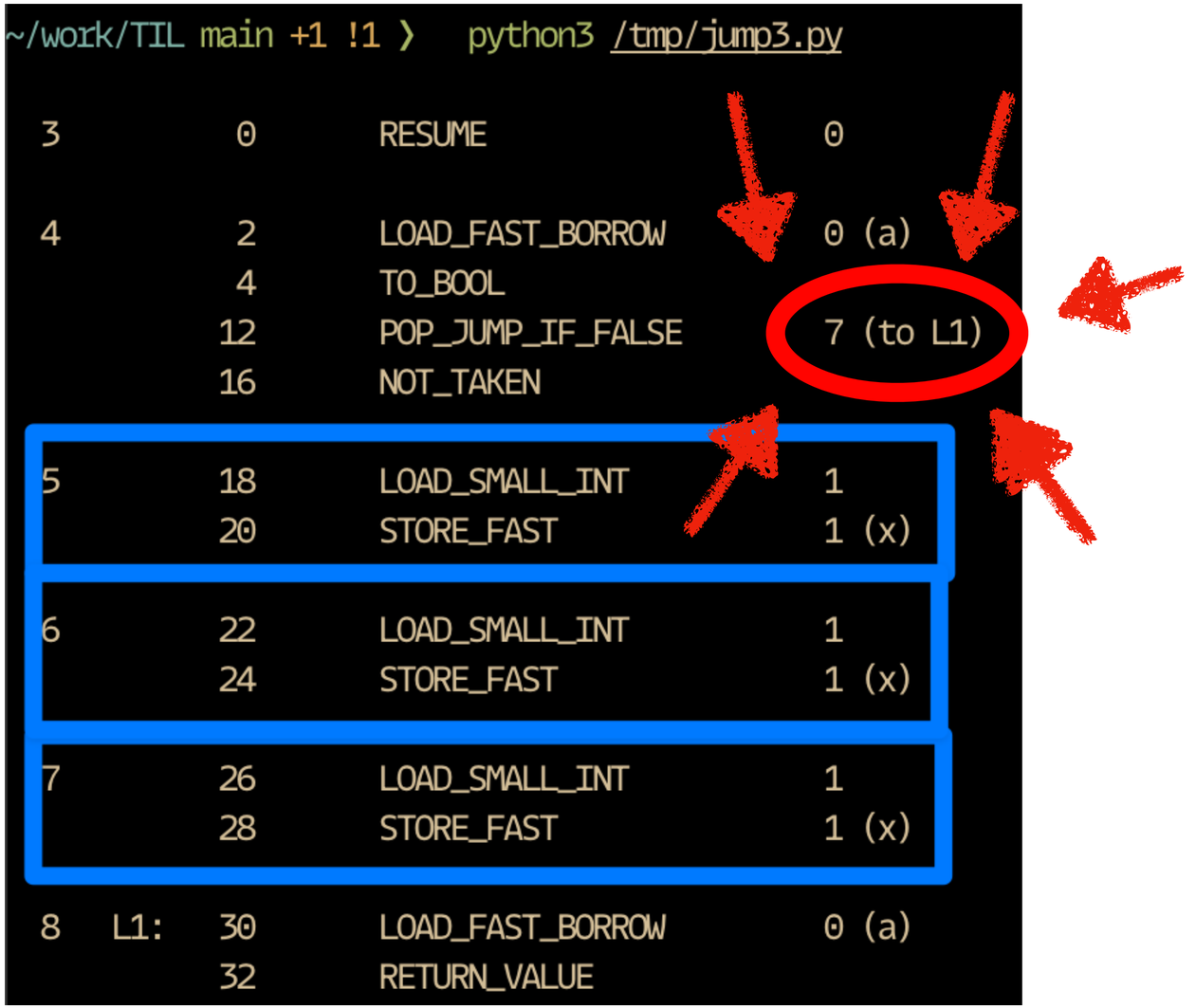
숫자는 점프 하는 상대 거리 (3칸 뒤로)
주소 표기에 비해 용량을 훨씬 아낄 수 있기 때문

RustPython 조건문 구현

```
~/work/TIL main +1 !1 ❯ cat /tmp/jump3.py
import dis

def f(a):
    if a:
        x = 1
        x = 1
        x = 1
    return a

dis.dis(f, show_offsets=True)
```



RustPython 조건문 구현: NOT_TAKEN

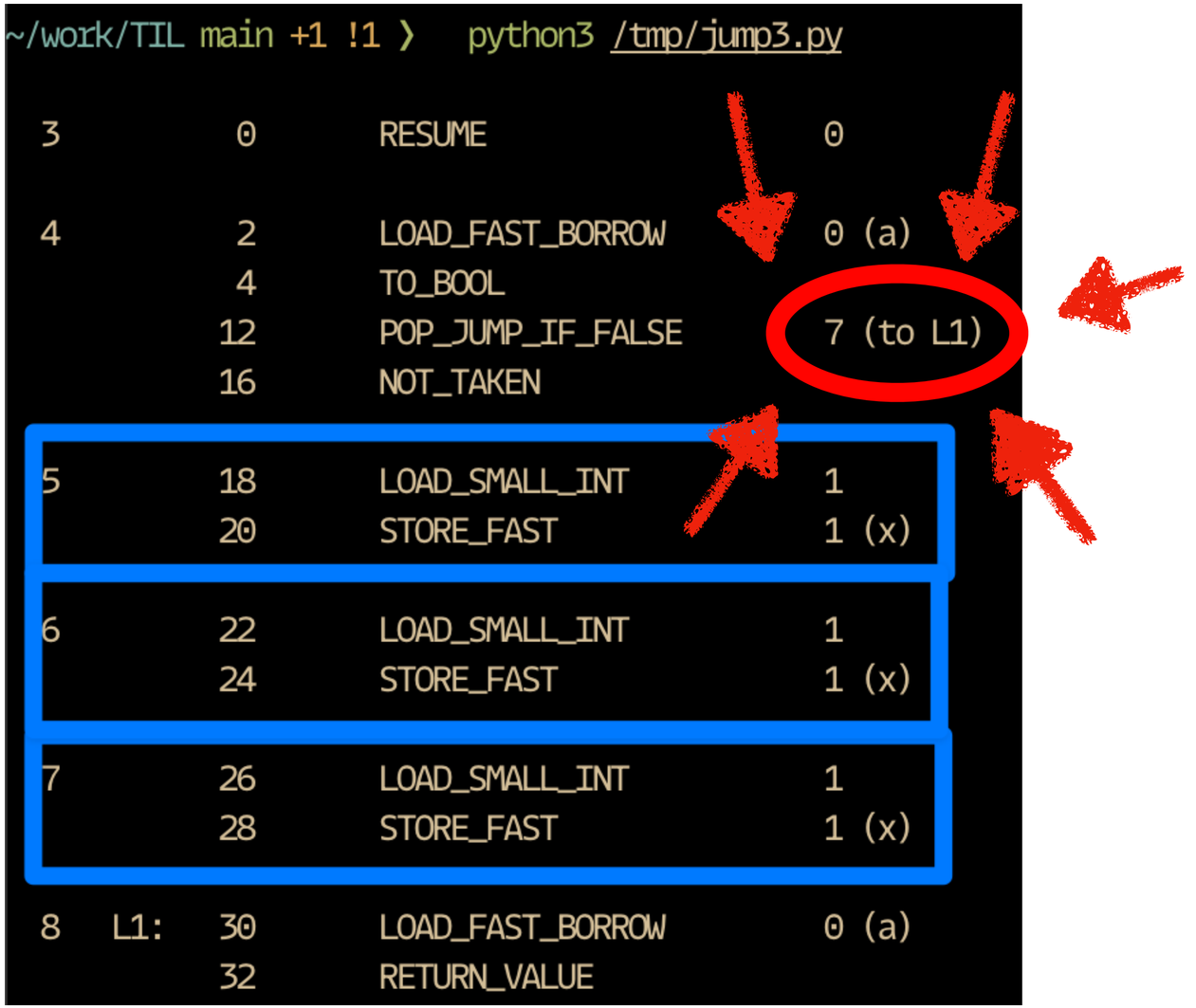
Q. NOT_TAKEN은 무엇일까

```rust
let is_forward = !self[last_ins.target].visited;

if is_forward {
    // Insert NOT_TAKEN after forward conditional jump.
    let not_taken = InstructionInfo {
        instr: Opcode::NotTaken.into(),
        arg: OpArg::new(0),
        target: BlockIdx::NULL,
        location: last_ins.location,
        end_location: last_ins.end_location,
        except_handler: None,
        lineno_override: last_ins.lineno_override,
    };

    self[block_idx].basicblock_addop(not_taken)?;
    return Ok(());
}
```

ada43c4f82b9df4e260f1368038c31cd55c4a61e/crates/codegen/src/ir.rs#L3266
https://github.com/RustPython/RustPython/blob/
ada43c4f82b9df4e260f1368038c31cd55c4a61e/crates/vm/src/frame.rs#L8268



RustPython 조건문 구현: NOT_TAKEN

Q. NOT_TAKEN은 무엇일까

crates/vm/src/frame.rs

```
Instruction::InstrumentedNotTaken => {
    if self.monitoring_mask & MonitoringEvent::BranchLeft.mask() != 0 {
        let not_taken_idx = self.lasti() as usize - 1;
        // Scan backwards past CACHE entries to find the branch instruction
        let mut branch_idx = not_taken_idx.saturating_sub(1);
        while branch_idx > 0
            && matches!(
                self.code.instructions.read_op(branch_idx),
                Instruction::Cache
            )
        {
            branch_idx -= 1;
        }
        let src_offset = (branch_idx as u32) * 2;
        let dest_offset = self.lasti() * 2;
        monitoring::fire_branch_left(vm, self.code, src_offset, dest_offset)?;
    }
    Ok(None)
}
```

RustPython 조건문 구현: 이슈

NOT_TAKEN, monitoring 관련된 버그를 찾음

```python
import sys
from sys import monitoring as m

def f(a):
    if a:
        pass

def left(*args):
    print("LEFT")
    return m.DISABLE

def right(*args):
    print("RIGHT")

m.use_tool_id(1, "test")
m.register_callback(1, m.events.BRANCH_LEFT, left)
m.register_callback(1, m.events.BRANCH_RIGHT, right)
m.set_local_events(1, f.__code__, m.events.BRANCH_LEFT | m.events.BRANCH_RIGHT)

print("--- f(True)");  f(True)    # LEFT
print("--- f(False)"); f(False)   # RIGHT
print("--- f(True)");  f(True)    # (출력 없음: 왼쪽은 꺼짐)
print("--- f(False)"); f(False)   # RIGHT
```

```
~/work/RustPython main *3 ❯ python3 /tmp/test.py
--- f(True)
LEFT
--- f(False)
RIGHT
--- f(True)
--- f(False)
RIGHT
~/work/RustPython main *3 ❯ ./target/debug/rustpython /tmp/test.py
--- f(True)
LEFT
--- f(False)
--- f(True)
--- f(False)
```

RustPython 조건문 구현: EXTENDED_ARG

CPython, RustPython에선
“POP_JUMP_IF_FALSE 3” 처럼 함 (숫자 칸은1byte)

그럼 값이 255가 넘으면 어떻게 하나요 ??

RustPython 조건문 구현: EXTENDED_ARG

```
bytes @12: 64 ff

POP_JUMP_IF_FALSE  255···
```

255줄 이하인 경우
그냥 씀

```
bytes @12..17: 45 01 45 01 64 d1

EXTENDED_ARG          숫자 칸 01 (  1)  쌓인 값 1
EXTENDED_ARG          숫자 칸 01 (  1)  쌓인 값 257
POP_JUMP_IF_FALSE     숫자 칸 d1 (209)  쌓인 값 66001
```

넘는 경우는

EXTENDED_ARG

읽은 명령	숫자 칸	쌓인 값
EXTENDED_ARG	1	1
EXTENDED_ARG	1	1 × 256 + 1 = 257
POP_JUMP_IF_FALSE	d1 (209)	257 × 256 + 209 = 66001

https://github.com/RustPython/RustPython/blob/ada43c4f82b9df4e260f1368038c31cd55c4a61e/crates/compiler-core/src/bytecode/oparg.rs#L4

RustPython 조건문 구현: 엣지 케이스

```
loop {
    // 1. 지금 크기대로 모든 명령의 위치를 셈
    for i in … { instr.i_offset = totsize; totsize += instr_size; }

    // 2. 점프 숫자를 채우고, 크기가 바뀌었는지 검사
    extended_arg_recompile = false;
    for i in … {
        info.arg = OpArg::new(oparg as u32);
        if info.instr_size() != i_size {
            extended_arg_recompile = true;
        }
    }

    // 3. 하나도 안 바뀌었으면 끝
    if !extended_arg_recompile { break; }
}
```

보통은 루프 안도는데,
아래같은 미묘한 케이스에
루프를 돌게됨

```
def f(a, b):
    if a: # 점프 A: 거짓이면 else로
        … # 컴파일 결과상 바이트코드 253칸
        # 점프 B: then 끝에서 else 건너뛰기
    else:
        … # 컴파일 결과상 바이트코드 300칸

    return a
```

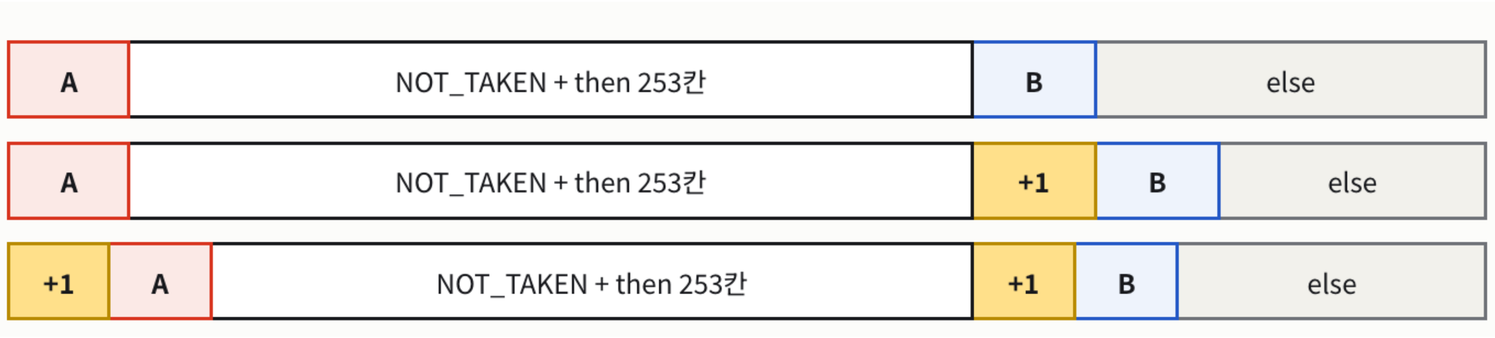
https://github.com/RustPython/RustPython/blob/ada43c4f82b9df4e260f1368038c31cd55c4a61e/crates/codegen/src/ir.rs#L1085

RustPython 조건문 구현: 엣지 케이스

쭉 읽다가 점프 B에 넘침(EXTENDED_ARG)을 적용하면
점프 A도 아슬아슬하게 넘침.
그러면 뒤쪽을 한 칸씩 다시 밀어줘야함

```python
def f(a, b):
    if a: # 점프 A: 거짓이면 else로
        ... # 컴파일 결과상 바이트코드 253칸
        # 점프 B: then 끝에서 else 건너뛰기
    else:
        ... # 컴파일 결과상 바이트코드 300칸

    return a
```



RustPython 조건문 구현: 엣지 케이스

```
loop {
    // 1. 지금 크기대로 모든 명령의 위치를 셈
    for i in … { instr.i_offset = totsize; totsize += instr_size; }

    // 2. 점프 숫자를 채우고, 크기가 바뀌었는지 검사
    extended_arg_recompile = false;
    for i in … {
        info.arg = OpArg::new(oparg as u32);
        if info.instr_size() != i_size {
            extended_arg_recompile = true;
        }
    }

    // 3. 하나도 안 바뀌었으면 끝
    if !extended_arg_recompile { break; }
}
```

https://github.com/RustPython/RustPython/blob/ada43c4f82b9df4e260f1368038c31cd55c4a61e/crates/codegen/src/ir.rs#L1085

RustPython 조건문 구현: 엣지 케이스

```rust
// crates/codegen/src/ir.rs
let mut totsize = 0i32;
for i in 0..instr_sequence.instr_used {
    let instr = &mut instr_sequence.instrs[i];
    instr.i_offset = totsize; // 이 명령의 위치
    let instr_size = instr.info.instr_size();
    totsize += instr_size as i32; // 다음 명령은 그만큼 뒤
}


// crates/compiler-core/src/bytecode/oparg.rs
// 숫자 크기로 칸 수를 정함
pub const fn instr_size(self) -> usize {
    (self.0 > 0xff) as usize
        + (self.0 > 0xff_ff) as usize
        + (self.0 > 0xff_ff_ff) as usize + 1
}
```

숫자	칸
0 ~ 255	1
256 ~ 65535	2
~ 16777215	3
그 이상	4

RustPython 조건문 구현: 엣지 케이스

```rust
// crates/codegen/src/ir.rs:931-965 EndAsyncFor
for i in 0..instr_sequence.instr_used {
    let i_size = instr_sequence.instrs[i].info.instr_size();  // 채우기 전 크기
    offset += i_size as i32;                                  // 이 명령 바로 다음 위치

    let opcode = instr_sequence.instrs[i].info.instr.expect_real();
    if opcode.has_jump() {
        let target = instr_sequence.instrs[i].i_target;
        let target_offset = instr_sequence.instrs[target as usize].i_offset;
        let info = &mut instr_sequence.instrs[i].info;
        let mut oparg = target_offset;
        if matches!(op, Instruction::EndAsyncFor) {  /* 특수 처리 생략 */
        } else if oparg < offset {
            oparg = offset - oparg; // 뒤로 가는 거리
        } else {
            oparg -= offset; // 앞으로 가는 거리
        }
        info.arg = OpArg::new(oparg as u32);
        // ********************************************************************
        if info.instr_size() != i_size { // 숫자를 넣었더니 길어졌는가 ?
            extended_arg_recompile = true;
        }
        // ********************************************************************
    }
}
```

```rust
loop {
    // 1. 지금 크기대로 모든 명령의 위치를 셈
    for i in … { instr.i_offset = totsize; totsize += instr_size; }

    // 2. 점프 숫자를 채우고, 크기가 바뀌었는지 검사
    extended_arg_recompile = false;
    for i in … {
        info.arg = OpArg::new(oparg as u32);
        if info.instr_size() != i_size {
            extended_arg_recompile = true;
        }
    }

    // 3. 하나도 안 바뀌었으면 끝
    if !extended_arg_recompile { break; }
}
```

근데 웬만해선 발생하지 않는 케이스

컴파일러는 미리 계산하고 싶다

x = 1

y = x + 2



Y = 3

컴파일러는 미리 계산하고 싶다

```
x = 1
if (a) { x = 5 }
y = x + 2
```



???

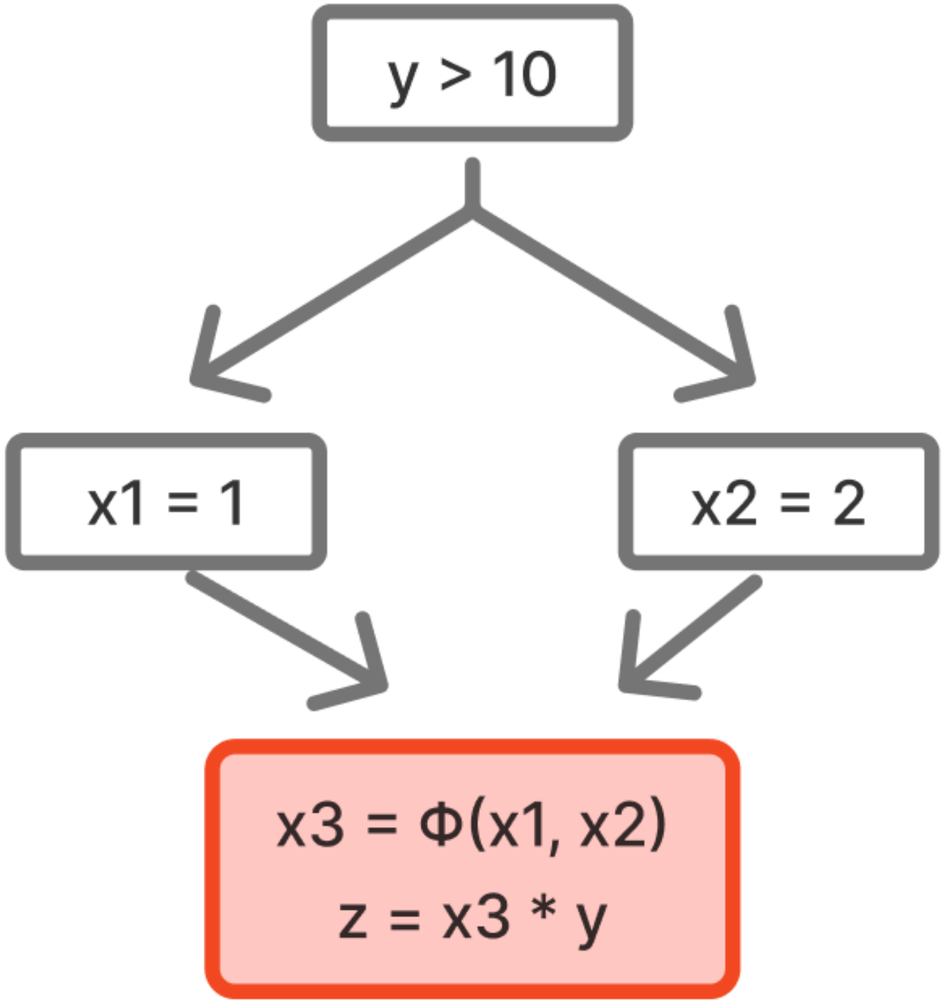
SSA: Static Single Assignment

```
x = 1
x = 5
y = x + 2
```



```
x1 = 1
x2 = 5
y1 = x2 + 2
```

SSA: Static Single Assignment

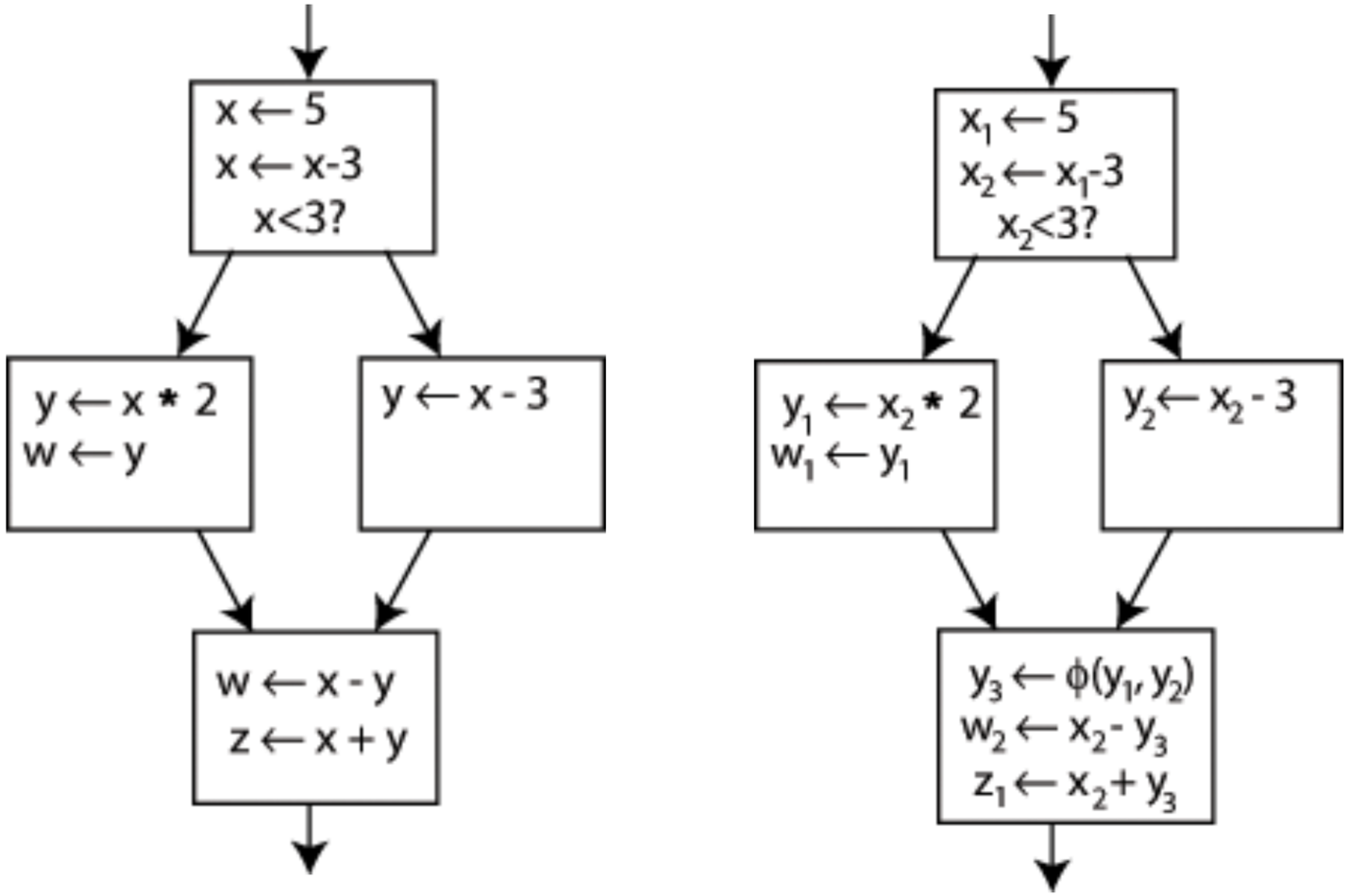


φ = Phi

Twenty-first letter in the Greek alphabet

'들어온 길에 따라 고른다'는 표시

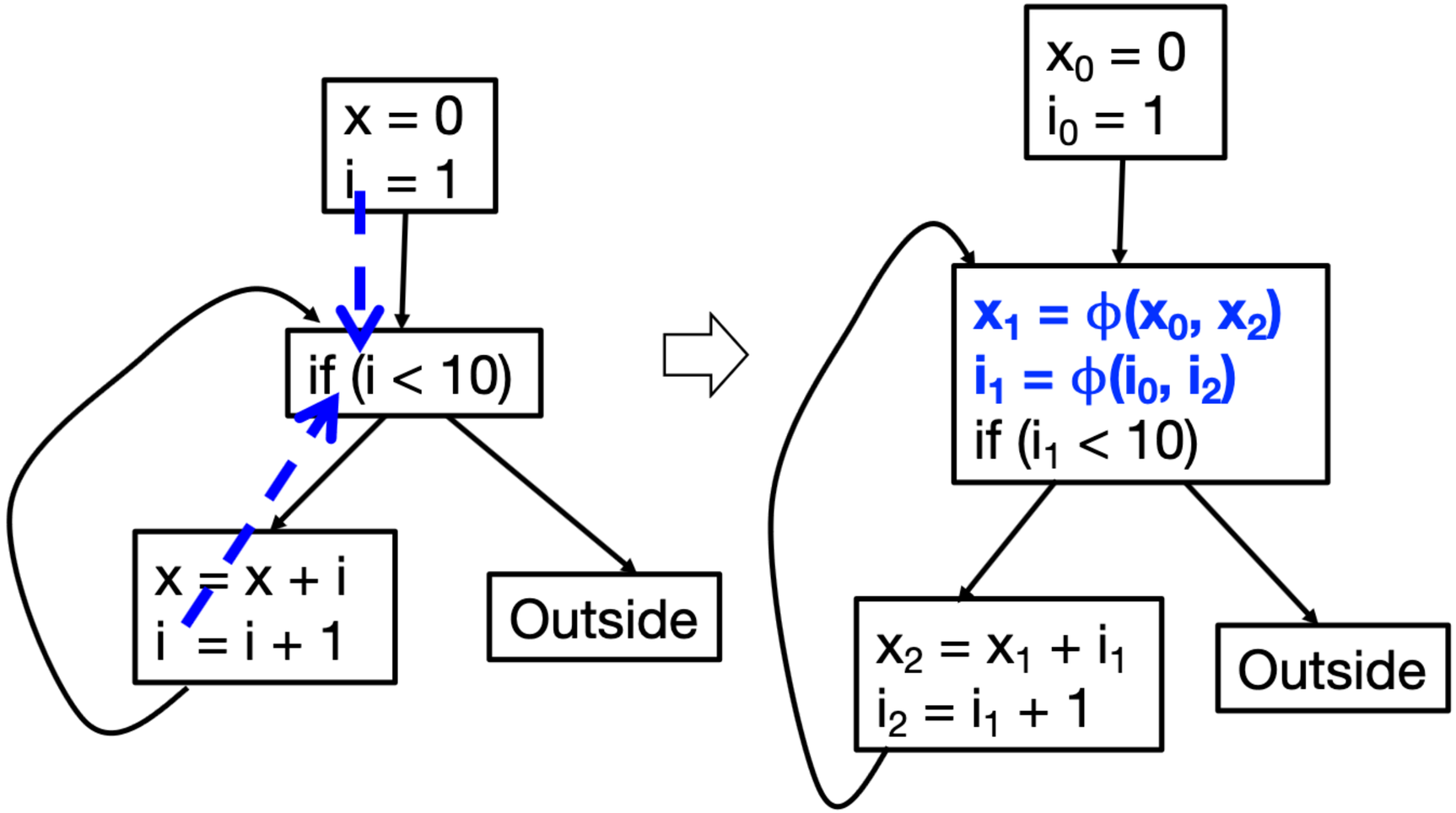
SSA: Static Single Assignment



https://en.wikipedia.org/wiki/Static_single-assignment_form

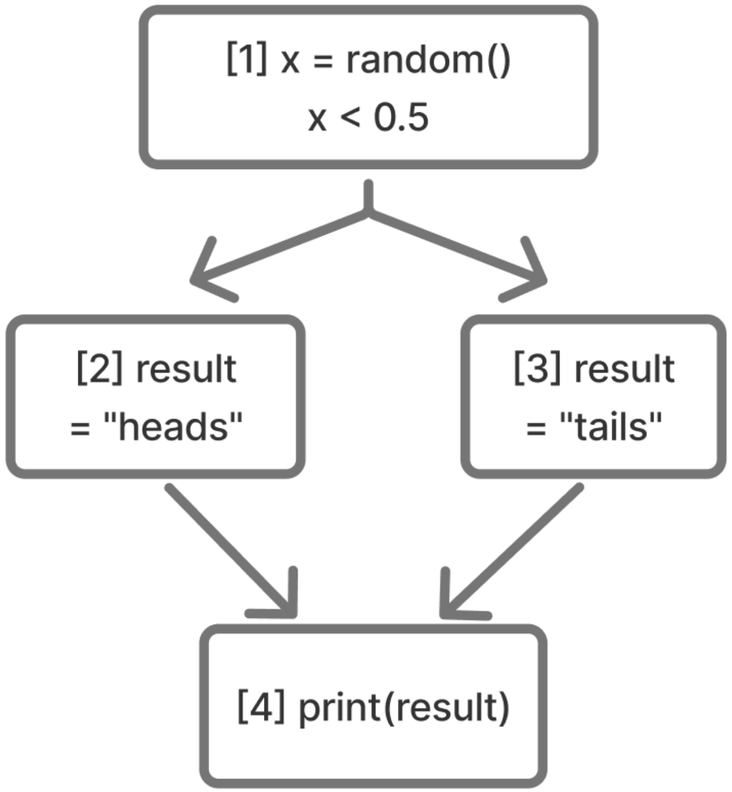
SSA: Static Single Assignment

```
x = 0
i = 1
while (i<10) {
    x = x+i
    i = i+1
}
```



https://www.csl.cornell.edu/courses/ece6775/pdf/lecture09.pdf

SSA: Dominance frontier



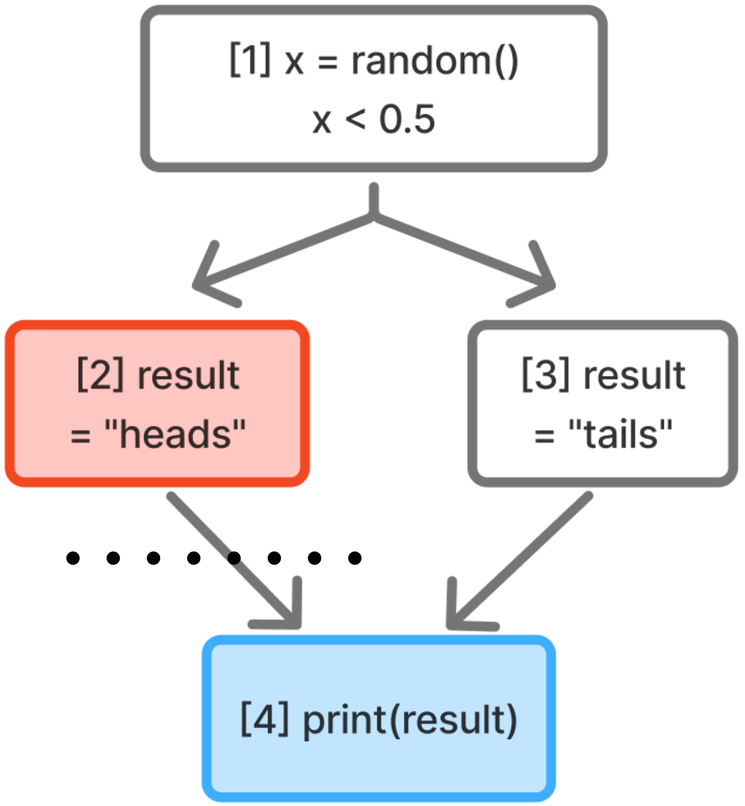

```
[1] x = random()
if x < 0.5
    [2] result = "heads"
else
    [3] result = "tails"
end
[4] print(result)
```

B에 가려면 반드시 A를 지나야함
= 'A가 B를 지배한다'

e.g. 1은 4를 지배함
2는 4를 지배하지 않음

https://en.wikipedia.org/wiki/Static_single-assignment_form

SSA: Dominance frontier

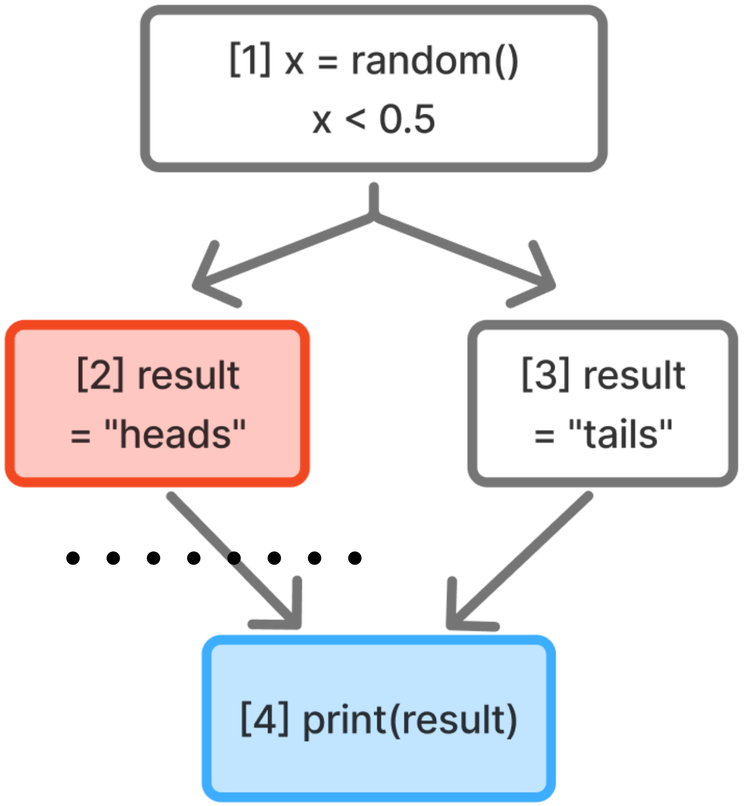



```
[1] x = random()
if x < 0.5
    [2] result = "heads"
else
    [3] result = "tails"
end
[4] print(result)
```

지배하면 ϕ가 필요 없다
ϕ가 필요한 곳 = 지배 경계

https://en.wikipedia.org/wiki/Static_single-assignment_form

SSA: Dominance frontier




```
[1] x = random()
if x < 0.5
    [2] result = "heads"
else
    [3] result = "tails"
end
[4] print(result)
```

지배하면 ϕ가 필요 없다
ϕ가 필요한 곳 = 지배 경계

https://en.wikipedia.org/wiki/Static_single-assignment_form

SSA: Static Single Assignment

1. 다 만든 뒤 계산하기 (Cytron 1991)
코드를 전부 만든 다음 변수가 정의된 블록의 지배 경계에 φ를 넣음
LLVM 같은 최적화 컴파일러

2. 읽으면서 만들기 (Braun 2013)
위에서부터 읽으며 기억 모르는 곳은 φ 자리만 두고 봉인 때 채움
Go 컴파일러, Cranelift

https://www.cs.cornell.edu/courses/cs6120/2025sp/blog/efficient-ssa/

SSA 사용 프로젝트 예시

- PyPy uses a linear SSA representation for traces in its JIT compiler.

  PyPy는 JIT 컴파일러에서 추적을 위해 선형 SSA 표현을 사용합니다.

- The LLVM Compiler Infrastructure uses SSA form for all scalar register values (everything except memory) in its primary code representation. SSA form is only eliminated once register allocation occurs, late in the compile process (often at link time).

  LLVM 컴파일러 인프라는 기본 코드 표현에서 모든 스칼라 레지스터 값(메모리를 제외한 모든 것)에 대해 SSA 형식을 사용합니다. SSA 형식은 레지스터 할당이 발생하고 컴파일 프로세스 후반(종종 링크 타임)에 제거됩니다.

- The GNU Compiler Collection (GCC) makes extensive use of SSA since version 4 (released in April 2005). The frontends generate "GENERIC" code that is then converted into "GIMPLE" code by the "gimplifier". High-level optimizations are then applied on the SSA form of "GIMPLE". The resulting optimized intermediate code is then translated into RTL, on which low-level optimizations are applied. The architecture-specific backends finally turn RTL into assembly language.

  GCC(GNU Compiler Collection)는 버전 4(2005년 4월 출시)부터 SSA를 광범위하게 사용합니다. 프런트엔드는 "GENERIC" 코드를 생성한 다음 "gimplifier"에 의해 "GIMPLE" 코드로 변환됩니다. 그런 다음 "GIMPLE"의 SSA 형식에 높은 수준의 최적화가 적용됩니다. 그 결과 최적화된 중간 코드는 낮은 수준의 최적화가 적용되는 RTL로 변환됩니다. 아키텍처별 백엔드는 마침내 RTL을 어셈블리 언어로 전환합니다.

- Go (1.7: for x86-64 architecture only; 1.8: for all supported architectures).[23][24]

  Go(1.7: x86-64 아키텍처 전용, 1.8: 지원되는 모든 아키텍처용).[23][24]